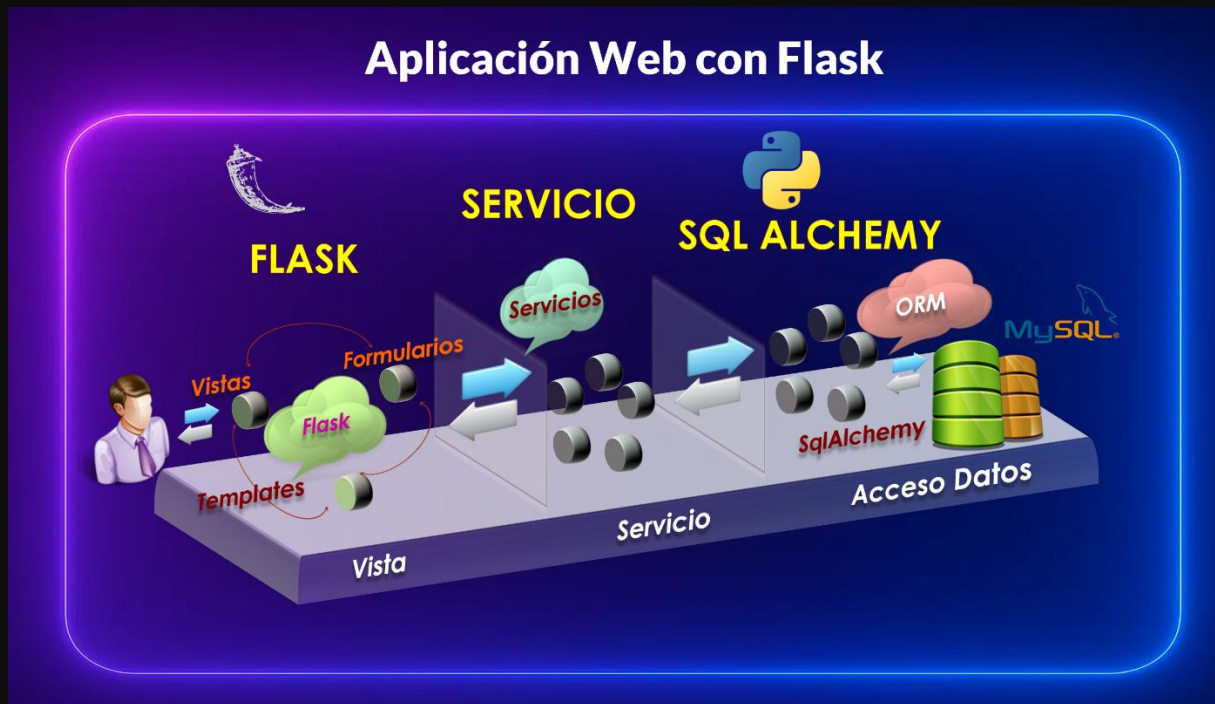




¡Perfecto! Empezamos con la **FASE 1: Preparación e instalación** ✅
 (Al terminar, me dices “**listo**” para continuar con la FASE 2.)

FASE 1: PREPARACIÓN E INSTALACIÓN

1) Pasos exactos / comandos

1. Crea la carpeta del proyecto:
 - o C:\Cursos\Flask\FlaskAcademy\flask_academy
2. Abre PyCharm y crea/abre el proyecto con esa carpeta.
3. En la **Terminal** integrada de PyCharm, instala las dependencias mínimas de esta fase:
4. `pip install flask python-dotenv`
5. Crea la estructura mínima de esta fase:
 - o C:\Cursos\Flask\FlaskAcademy\flask_academy\app.py
 - o C:\Cursos\Flask\FlaskAcademy\flask_academy\.env ← (sin extensión extra)
 - o C:\Cursos\Flask\FlaskAcademy\flask_academy\templates\index.html

2) Archivos a crear o modificar (con rutas completas)

A) (Nuevo) C:\Cursos\Flask\FlaskAcademy\flask_academy\app.py

```
import os
from flask import Flask, render_template
```

```

from dotenv import load_dotenv

# Carga variables de entorno desde .env (para usar SECRET_KEY ahora
# y más tarde las credenciales de la base de datos).
load_dotenv()

def create_app():
    app = Flask(__name__)

    # SECRET_KEY se usa para sesiones y protección CSRF (lo necesitaremos con Flask-
    WTF).
    app.config["SECRET_KEY"] = os.getenv("SECRET_KEY", "dev-secret-change-me")

    # Ruta principal (aún sin lógica de negocio; solo renderiza la plantilla base).
    @app.route("/")
    def index():
        return render_template("index.html")

    return app

# Soporte para ejecutar en PyCharm con botón "Run"
if __name__ == "__main__":
    app = create_app()
    app.run(debug=True)

```

B) (Nuevo) C:\Cursos\Flask\FlaskAcademy\flask_academy\.env

```

# ⚠ IMPORTANTE: este archivo debe llamarse exactamente ".env" (sin .txt, .py ni otra
extensión)
# Clave para sesiones y formularios (CSRF). Cambiar por una segura al finalizar el
proyecto.
SECRET_KEY=super-clave-ultra-secreta-para-desarrollo

```

⚠ Advertencia solicitada: **NO** agregues ninguna extensión al archivo .env (nada de .env.txt, .env.py, etc.). Debe ser **exactamente** .env.

C) (Nuevo) C:\Cursos\Flask\FlaskAcademy\flask_academy\templates\index.html

```

<!doctype html>
<html lang="es" data-bs-theme="dark">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Flask Academy</title>

    <!-- Bootstrap 5 desde CDN, sin archivos locales -->
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet"
      integrity="sha384-
QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
      crossorigin="anonymous"
    >

```

```

</head>
<body>
  <div class="container py-5">
    <header class="mb-4">
      <h1 class="text-center text-warning fw-bold display-5">🔥 Flask Academy</h1>
    </header>

    <main>
      <div class="card border-0 shadow">
        <div class="card-body">
          <p class="mb-0">
            Bienvenido/a. La aplicación Flask está funcionando correctamente. 🟢
            En las siguientes fases conectaremos MySQL y construiremos el CRUD
            completo.
          </p>
        </div>
      </div>
    </main>
  </div>

  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
  integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
  crossorigin="anonymous">
  </script>
</body>
</html>

```

Requisitos cumplidos:

- **Modo oscuro:** `data-bs-theme="dark"` en `<html>`.
- **Sin CSS personalizado:** solo clases de Bootstrap (`text-warning` para título amarillo, `text-center`).
- **Título exacto:** “Flask Academy” centrado, color amarillo, con ícono 🔥.

3) Explicación breve de cada archivo

- **app.py:** Punto de entrada de la app. Crea la instancia de Flask, carga la `SECRET_KEY` desde `.env` (usada para sesiones/CSRF), y define la ruta `/` que renderiza `index.html`. No contiene lógica de negocio (esa vivirá en `services/` en fases posteriores).
- **.env:** Guarda variables sensibles y de configuración (p.ej., `SECRET_KEY` y, en la FASE 2, credenciales de MySQL). Mantener **sin extensión** y fuera del control de versiones en proyectos reales.
- **templates/index.html:** Plantilla base con **Bootstrap** desde CDN en **modo oscuro**. Solo muestra una tarjeta mínima para verificar que Flask renderiza correctamente.

4) Prueba rápida

- En PyCharm, ejecuta `app.py` (Run ▶).
 - Abre el navegador en: `http://127.0.0.1:5000/`
 - Debes ver:
 - Título 🚩 **Flask Academy** centrado y amarillo.
 - Una tarjeta indicando que Flask está funcionando.
-

5) Solución a errores típicos

- **ModuleNotFoundError: No module named 'flask'**
→ Asegúrate de haber corrido: `pip install flask` (en la terminal del intérprete que usa PyCharm).
 - **Se ve una página en blanco o error de plantilla**
→ Verifica que la carpeta se llame exactamente `templates` y que `index.html` esté dentro de `flask_academy/templates/`.
 - **El `.env` no carga** (por ahora solo usamos `SECRET_KEY`):
→ Confirma que el archivo sea `.env` exacto, sin extensión extra, y ubicado en `flask_academy/.env`.
→ Si cambiaste la clave, reinicia la app para que se recargue la variable.
-

6) Qué sigue

Con la base corriendo, en la **FASE 2** conectaremos **MySQL** (crearemos `extensions.py`, configuraremos la URL de conexión a partir del `.env`, y dejaremos lista la app para migraciones con SQLAlchemy/Migrate).

¿Listo para continuar? Cuando lo verifiques en tu navegador, respóndeme “**listo**” y pasamos a la **FASE 2**.

Dijiste:

listo pasamos a la fase 2

ChatGPT dijo:

Excelente 🎯

Pasemos a la **FASE 2: Conexión con MySQL**

FASE 2: CONEXIÓN CON MYSQL

En esta fase conectaremos **Flask** con **MySQL** utilizando **SQLAlchemy** y **Flask-Migrate**, de forma profesional y modular.

Toda la configuración quedará centralizada en el archivo `.env` y en un nuevo archivo `extensions.py`.

1 Instalación de dependencias

Ejecuta en la terminal de **PyCharm** dentro del proyecto:

```
pip install flask-sqlalchemy flask-migrate pymysql
```

Estas librerías sirven para:

- `flask-sqlalchemy`: ORM para interactuar con MySQL mediante objetos Python.
 - `flask-migrate`: Control de versiones de la base de datos.
 - `pymysql`: Conector de Python con MySQL.
-

2 Crear o modificar archivos

A) (Nuevo) `C:\Cursos\Flask\FlaskAcademy\flask_academy\extensions.py`

```
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate

# Inicializamos las extensiones de forma global
db = SQLAlchemy()
migrate = Migrate()
```

💡 Este archivo separa la inicialización de las extensiones de la app principal, facilitando la reutilización y evitando ciclos de importación.

B) (Modificar) `C:\Cursos\Flask\FlaskAcademy\flask_academy\app.py`

Actualizamos el archivo para incluir la configuración de base de datos.

```
import os
from flask import Flask, render_template
from dotenv import load_dotenv
```

```

from extensions import db, migrate

# Carga variables de entorno desde .env
load_dotenv()

def create_app():
    app = Flask(__name__)

    # Clave secreta para formularios y sesiones
    app.config["SECRET_KEY"] = os.getenv("SECRET_KEY", "dev-secret-change-me")

    # Configuración de conexión a MySQL
    usuario = os.getenv("DB_USER")
    password = os.getenv("DB_PASSWORD")
    servidor = os.getenv("DB_HOST")
    puerto = os.getenv("DB_PORT")
    base_datos = os.getenv("DB_NAME")

    app.config["SQLALCHEMY_DATABASE_URI"] = (
        f"mysql+pymysql://{usuario}:{password}@{servidor}:{puerto}/{base_datos}"
    )
    app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

    # Inicializamos las extensiones
    db.init_app(app)
    migrate.init_app(app, db)

    # Ruta principal
    @app.route("/")
    def index():
        return render_template("index.html")

    return app

if __name__ == "__main__":
    app = create_app()
    app.run(debug=True)

```

C) (Modificar) C:\Cursos\Flask\FlaskAcademy\flask_academy\.env

Agrega ahora las variables de conexión (puedes mantener la SECRET_KEY existente):

```

SECRET_KEY=super-clave-ultra-secreta-para-desarrollo
DB_USER=root
DB_PASSWORD=admin
DB_HOST=localhost
DB_PORT=3306
DB_NAME=flask_academy_db

```

⚠ Nota importante: No agregues comillas ni espacios alrededor de los valores. Este archivo debe seguir llamándose **.env** sin extensión.

3 Crear base de datos en MySQL (si no existe)

Abre **MySQL Workbench** y ejecuta:

```
CREATE DATABASE flask_academy_db;
```

4 Verificación de conexión

Aún no hemos creado modelos, pero podemos probar la conexión ejecutando la app:

```
python app.py
```

Si todo está correcto:

- Flask iniciará sin errores.
- Verás el mensaje de que el servidor corre en `http://127.0.0.1:5000/`.
- Podrás abrir la página con el título 🦄 **Flask Academy**.

5 Solución a errores comunes

Error	Causa probable	Solución
<code>ModuleNotFoundError: No module named 'pymysql'</code>	No se instaló el conector	Ejecuta <code>pip install pymysql</code>
<code>Access denied for user 'root'@'localhost'</code>	Credenciales incorrectas	Revisa usuario y contraseña en <code>.env</code>
<code>Unknown database 'flask_academy_db'</code>	No se creó la base de datos	Ejecuta el <code>CREATE DATABASE</code> en MySQL
<code>sqlalchemy.exc.OperationalError</code>	Puerto o host incorrecto	Verifica <code>DB_HOST</code> y <code>DB_PORT</code> en <code>.env</code>

6 Explicación rápida

- **extensions.py** → Inicializa `db` y `migrate` para que sean compartidos en toda la app.
- **app.py** → Configura la conexión leyendo variables desde `.env`.
- **.env** → Almacena los datos sensibles.
- **mysql+pymysql://...** → Es el formato estándar de conexión que usa SQLAlchemy para MySQL.

✅ Con esto ya tenemos Flask conectado correctamente a MySQL y listo para crear tablas con migraciones en la siguiente fase.

Cuando hayas verificado que la app inicia sin errores y la base de datos existe, responde “**listo**” y pasaremos a la **FASE 3: CREACIÓN DEL MODELO**.

Dijiste:

listo vamos a la fase 3

ChatGPT dijo:

Perfecto 🚀

Pasemos ahora a la **FASE 3: CREACIÓN DEL MODELO**

🧩 FASE 3: CREACIÓN DEL MODELO

En esta fase definiremos la estructura de la tabla `curso` mediante una clase modelo en SQLAlchemy, y usaremos **Flask-Migrate** para generar y aplicar las migraciones en MySQL.

1 Archivos a crear o modificar

A) (Nuevo) C:\Cursos\Flask\FlaskAcademy\flask_academy\models.py

```
from extensions import db

# Clase que representa la tabla 'curso' en la base de datos
class Curso(db.Model):
    __tablename__ = "curso"

    id_curso = db.Column(db.Integer, primary_key=True, autoincrement=True)
    nombre = db.Column(db.String(100), nullable=False)
    instructor = db.Column(db.String(100), nullable=False)
    duracion = db.Column(db.Numeric(5, 2), nullable=False)

    def __repr__(self):
        return f"<Curso {self.nombre}>"
```

Explicación:

- `__tablename__`: define el nombre real de la tabla en MySQL.

- `id_curso`: clave primaria autoincremental.
- `nombre`, `instructor`: columnas tipo cadena obligatorias.
- `duracion`: tipo numérico con 2 decimales, ideal para horas o duración en general.
- `__repr__`: representación útil al imprimir objetos (p. ej., en consola).

B) (Modificar) `C:\Cursos\Flask\FlaskAcademy\flask_academy\app.py`

Agregaremos la importación del modelo para que Flask-Migrate lo detecte al crear las migraciones.

```
import os
from flask import Flask, render_template
from dotenv import load_dotenv
from extensions import db, migrate
from models import Curso # 🐱 Importamos el modelo aquí

load_dotenv()

def create_app():
    app = Flask(__name__)
    app.config["SECRET_KEY"] = os.getenv("SECRET_KEY", "dev-secret-change-me")

    usuario = os.getenv("DB_USER")
    password = os.getenv("DB_PASSWORD")
    servidor = os.getenv("DB_HOST")
    puerto = os.getenv("DB_PORT")
    base_datos = os.getenv("DB_NAME")

    app.config["SQLALCHEMY_DATABASE_URI"] = (
        f"mysql+pymysql://{usuario}:{password}@{servidor}:{puerto}/{base_datos}"
    )
    app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

    db.init_app(app)
    migrate.init_app(app, db)

    @app.route("/")
    def index():
        return render_template("index.html")

    return app

if __name__ == "__main__":
    app = create_app()
    app.run(debug=True)
```

2 Generar migraciones (una vez creados los archivos anteriores)

Ejecuta los siguientes comandos desde la carpeta raíz del proyecto (C:\Cursos\Flask\FlaskAcademy\flask_academy):

```
flask db init
flask db migrate -m "Creación de la tabla curso"
flask db upgrade
```

Qué hace cada comando:

- `flask db init` → crea la carpeta `migrations/` (solo una vez por proyecto).
- `flask db migrate` → analiza los modelos y genera un script de migración.
- `flask db upgrade` → aplica los cambios a la base de datos real.

💡 Si obtienes error “flask no se reconoce como comando”, ejecuta:

```
python -m flask db init
python -m flask db migrate -m "Creación de la tabla curso"
python -m flask db upgrade
```

3 Confirmar creación en MySQL Workbench

1. Abre tu base de datos `flask_academy_db`.
 2. Expande las tablas y verifica que exista una tabla llamada **curso**.
 3. Abre y revisa las columnas:
 - `id_curso` (PK, int, autoincrement)
 - `nombre` (varchar 100)
 - `instructor` (varchar 100)
 - `duracion` (decimal 5,2)
-

4 Prueba rápida

Ejecuta nuevamente el proyecto:

```
python app.py
```

La app debe correr igual que antes.

Por ahora no hay cambios visuales, pero internamente la tabla ya está lista en MySQL para el CRUD.

5 Solución a errores típicos

Error	Causa probable	Solución
No module named 'models'	Import mal ubicado	Verifica que <code>models.py</code> esté en la raíz del proyecto Flask
Target database is not up to date	Migraciones pendientes	Ejecuta <code>flask db upgrade</code>
<code>flask db migrate</code> no crea nada	Olvidaste importar el modelo en <code>app.py</code>	Asegúrate de tener <code>from models import Curso</code>

6 Explicación breve del flujo

- **SQLAlchemy** traduce la clase `Curso` en una tabla SQL.
- **Flask-Migrate** detecta automáticamente los modelos para mantener la base sincronizada.
- La separación en `models.py` hace que la aplicación sea más modular y escalable.

✓ Con esto finalizamos la **FASE 3**.

La aplicación ya tiene su estructura de base de datos completamente funcional.

Cuando confirmes que la tabla `curso` aparece correctamente en **MySQL Workbench**, escribe “listo” y avanzaremos a la **FASE 4: CASO DE LISTAR (READ)**.

Dijiste:

listo avancemos a la fase 4

ChatGPT dijo:

Excelente 🍀

Entramos ahora a la **FASE 4: CASO DE LISTAR (READ)**

Aquí mostraremos los cursos almacenados en la base de datos directamente en la página principal `index.html`, aplicando el patrón de **capa de servicios** para mantener el código limpio y modular.

🍀 FASE 4: CASO DE LISTAR (READ)

1 Objetivo de la fase

- Crear un archivo `curso_service.py` dentro de la carpeta `services/` que contenga la lógica para **obtener todos los cursos** desde la base de datos.
- Modificar `app.py` para delegar la lógica de acceso a datos al servicio.
- Mostrar los cursos en una **tabla Bootstrap** dentro de `index.html`.

2 Archivos a crear o modificar

A) (Nuevo)

 C:\Cursos\Flask\FlaskAcademy\flask_academy\services\curso_service.py

```
from extensions import db
from models import Curso

# =====
# 🐡 Función para obtener todos los cursos
# =====
def obtener_todos():
    """
    Retorna una lista con todos los cursos almacenados en la base de datos.
    """
    return Curso.query.all()
```

🧠 Explicación:

- La lógica de acceso a datos se coloca aquí, **no en app.py**.
- Esto permite separar la capa de presentación (rutas y vistas) de la lógica de negocio.
- `Curso.query.all()` usa SQLAlchemy para recuperar todos los registros.

B) (Modificar)

 C:\Cursos\Flask\FlaskAcademy\flask_academy\app.py

```
import os
from flask import Flask, render_template
from dotenv import load_dotenv
from extensions import db, migrate
from models import Curso
from services import curso_service # 👉 Importamos la capa de servicios

load_dotenv()

def create_app():
    app = Flask(__name__)
    app.config["SECRET_KEY"] = os.getenv("SECRET_KEY", "dev-secret-change-me")
```

```

usuario = os.getenv("DB_USER")
password = os.getenv("DB_PASSWORD")
servidor = os.getenv("DB_HOST")
puerto = os.getenv("DB_PORT")
base_datos = os.getenv("DB_NAME")

app.config["SQLALCHEMY_DATABASE_URI"] = (
    f"mysql+pymysql://{usuario}:{password}@{servidor}:{puerto}/{base_datos}"
)
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

db.init_app(app)
migrate.init_app(app, db)

# Ruta principal que lista los cursos
@app.route("/")
def index():
    cursos = curso_service.obtener_todos()
    return render_template("index.html", cursos=cursos)

return app

if __name__ == "__main__":
    app = create_app()
    app.run(debug=True)

```

💡 Nota:

`index()` ahora solo se encarga de **enviar datos al template**, mientras que la obtención de datos se maneja desde `curso_service.py`.

C) (Modificar)

 C:\Cursos\Flask\FlaskAcademy\flask_academy\templates\index.html

Actualizamos la plantilla para mostrar los cursos en una tabla con **modo oscuro y diseño limpio**:

```

<!doctype html>
<html lang="es" data-bs-theme="dark">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Flask Academy</title>

    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet"
      integrity="sha384-
QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH"
      crossorigin="anonymous"

```

```

>
</head>
<body>
  <div class="container py-5">
    <header class="mb-4">
      <h1 class="text-center text-warning fw-bold display-5">🦄 Flask Academy</h1>
    </header>

    <main>
      <div class="card border-0 shadow">
        <div class="card-body">
          {% if cursos %}
            <table class="table table-dark table-hover align-middle">
              <thead>
                <tr class="text-warning">
                  <th>ID</th>
                  <th>Curso</th>
                  <th>Instructor</th>
                  <th>Duración (hrs)</th>
                </tr>
              </thead>
              <tbody>
                {% for curso in cursos %}
                  <tr>
                    <td>{{ curso.id_curso }}</td>
                    <td>{{ curso.nombre }}</td>
                    <td>{{ curso.instructor }}</td>
                    <td>{{ curso.duracion }}</td>
                  </tr>
                {% endfor %}
              </tbody>
            </table>
          {% else %}
            <p class="text-muted mb-0">No hay cursos registrados aún.</p>
          {% endif %}
        </div>
      </div>
    </main>
  </div>

  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
  integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
  crossorigin="anonymous">
  </script>
</body>
</html>

```

3 Prueba rápida

1. En **MySQL Workbench**, agrega algunos registros de prueba:
2. `INSERT INTO curso (nombre, instructor, duracion)`

3. VALUES
4. ('Python desde Cero', 'María López', 20.0),
5. ('Flask Intermedio', 'Carlos Pérez', 15.5),
6. ('Bases de Datos MySQL', 'Laura Torres', 12.0);
7. Ejecuta el proyecto:
8. python app.py
9. Abre tu navegador en
 http://127.0.0.1:5000/

Deberías ver una tabla con los cursos cargados desde MySQL .

Solución a errores comunes

Error	Causa probable	Solución
AttributeError: 'NoneType' object has no attribute 'query'	No se importó correctamente el modelo o db no se inicializó	Revisa <code>from models import Curso</code> y <code>db.init_app(app)</code>
jinja2.exceptions.UndefinedError: 'cursos' is undefined	No se pasó la variable al template	Asegúrate de que <code>render_template("index.html", cursos=cursos)</code> esté en <code>app.py</code>
La tabla aparece vacía	No hay datos en MySQL	Inserta registros manualmente desde Workbench

Explicación breve

- `curso_service.obtener_todos()` encapsula el acceso a la base de datos.
- `index()` en `app.py` solo se ocupa de preparar los datos para la vista.
- `index.html` muestra una tabla estilizada con **Bootstrap modo oscuro**.
- La estructura modular permite mantener un código **limpio y escalable**.

 Con esto completamos la **FASE 4: READ (Listar cursos)**.

El sistema ya puede mostrar los cursos existentes de forma elegante en la interfaz web.

Cuando verifiques que los cursos aparecen correctamente en tu navegador, responde “**listo**” y avanzaremos a la **FASE 5: CASO AGREGAR (CREATE)**.

Dijiste:

listo avancemos a la fase 5

ChatGPT dijo:

Perfecto 🙌

Pasemos a una de las partes más interesantes:

la **FASE 5: CASO AGREGAR (CREATE)** 🚀

Aquí implementaremos la funcionalidad para **agregar nuevos cursos** mediante un formulario con **Flask-WTF** y **Bootstrap**, respetando el patrón de arquitectura por capas.

🧩 FASE 5: CASO AGREGAR (CREATE)

1 Objetivo

- Instalar **Flask-WTF** para trabajar con formularios seguros.
 - Agregar una barra de navegación en modo oscuro con Bootstrap.
 - Crear un formulario (`agregar_curso.html`) para añadir nuevos cursos.
 - Implementar la lógica de inserción en `curso_service.py`.
 - Verificar que al guardar, se redirija al listado y se actualice automáticamente.
-

2 Instalación

Ejecuta el siguiente comando en la terminal de **PyCharm**:

```
pip install flask-wtf
```

3 Archivos a crear o modificar

A) (Nuevo)

📄 C:\Cursos\Flask\FlaskAcademy\flask_academy\templates\agregar_curso.html

```
<!doctype html>
<html lang="es" data-bs-theme="dark">
  <head>
```



```

<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Agregar Curso - Flask Academy</title>

<link
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
  rel="stylesheet"
  integrity="sha384-
QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH"
  crossorigin="anonymous"
>
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark mb-4">
    <div class="container">
      <a class="navbar-brand text-warning fw-bold" href="/">🦄 Flask Academy</a>
      <div>
        <a href="/" class="btn btn-outline-warning btn-sm me-2">Listado</a>
        <a href="/agregar" class="btn btn-warning btn-sm">Agregar Curso</a>
      </div>
    </div>
  </nav>

  <div class="container">
    <div class="card border-0 shadow">
      <div class="card-body">
        <h2 class="text-warning mb-4">+ Agregar Nuevo Curso</h2>

        <form method="POST">
          {{ form.hidden_tag() }}

          <div class="mb-3">
            {{ form.nombre.label(class="form-label") }}
            {{ form.nombre(class="form-control", placeholder="Nombre del curso") }}
          </div>

          <div class="mb-3">
            {{ form.instructor.label(class="form-label") }}
            {{ form.instructor(class="form-control", placeholder="Instructor") }}
          </div>

          <div class="mb-3">
            {{ form.duracion.label(class="form-label") }}
            {{ form.duracion(class="form-control", placeholder="Duración en horas") }}
          </div>

          <button type="submit" class="btn btn-warning">Guardar</button>
          <a href="/" class="btn btn-secondary ms-2">Cancelar</a>
        </form>
      </div>
    </div>
  </div>

  <script

```

```
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
  integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
  crossorigin="anonymous">
</script>
</body>
</html>
```

B) (Nuevo)

 C:\Cursos\Flask\FlaskAcademy\flask_academy\forms.py

```
from flask_wtf import FlaskForm
from wtforms import StringField, DecimalField, SubmitField
from wtforms.validators import DataRequired, Length, NumberRange

class CursoForm(FlaskForm):
    nombre = StringField("Nombre del Curso", validators=[
        DataRequired(),
        Length(min=3, max=100, message="El nombre debe tener entre 3 y 100 caracteres")
    ])
    instructor = StringField("Instructor", validators=[
        DataRequired(),
        Length(min=3, max=100, message="El nombre del instructor debe tener entre 3 y
100 caracteres")
    ])
    duracion = DecimalField("Duración (horas)", validators=[
        DataRequired(),
        NumberRange(min=1, max=999, message="Debe ser un número entre 1 y 999 horas")
    ])
    submit = SubmitField("Guardar")
```

💡 Explicación:

- `FlaskForm` es la base de los formularios seguros.
 - `hidden_tag()` agrega el token CSRF automáticamente.
 - Validadores garantizan que los campos sean válidos antes de guardar.
-

C) (Modificar)

 C:\Cursos\Flask\FlaskAcademy\flask_academy\services\curso_service.py

Agregamos la función para insertar un nuevo curso.

```
from extensions import db
from models import Curso
```

```
def obtener_todos():
    return Curso.query.all()

# =====
# 🌸 Función para agregar un curso nuevo
# =====
def agregar_curso(nombre, instructor, duracion):
    """
    Crea un nuevo curso en la base de datos.
    """
    nuevo_curso = Curso(nombre=nombre, instructor=instructor, duracion=duracion)
    db.session.add(nuevo_curso)
    db.session.commit()
```

D) (Modificar)

 C:\Cursos\Flask\FlaskAcademy\flask_academy\app.py

Agregamos la ruta /agregar y la barra de navegación a index.html.

```
import os
from flask import Flask, render_template, redirect, url_for, flash
from dotenv import load_dotenv
from extensions import db, migrate
from models import Curso
from services import curso_service
from forms import CursoForm # 🖱️ nuevo

load_dotenv()

def create_app():
    app = Flask(__name__)
    app.config["SECRET_KEY"] = os.getenv("SECRET_KEY", "dev-secret-change-me")

    usuario = os.getenv("DB_USER")
    password = os.getenv("DB_PASSWORD")
    servidor = os.getenv("DB_HOST")
    puerto = os.getenv("DB_PORT")
    base_datos = os.getenv("DB_NAME")

    app.config["SQLALCHEMY_DATABASE_URI"] = (
        f"mysql+pymysql://{usuario}:{password}@{servidor}:{puerto}/{base_datos}"
    )
    app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

    db.init_app(app)
    migrate.init_app(app, db)

    @app.route("/")
    def index():
        cursos = curso_service.obtener_todos()
        return render_template("index.html", cursos=cursos)
```

```
# Nueva ruta para agregar curso
@app.route("/agregar", methods=["GET", "POST"])
def agregar():
    form = CursoForm()
    if form.validate_on_submit():
        curso_service.agregar_curso(
            form.nombre.data,
            form.instructor.data,
            form.duracion.data
        )
        flash("✅ Curso agregado exitosamente", "success")
        return redirect(url_for("index"))
    return render_template("agregar_curso.html", form=form)

return app

if __name__ == "__main__":
    app = create_app()
    app.run(debug=True)
```

E) (Modificar)

 C:\Cursos\Flask\FlaskAcademy\flask_academy\templates\index.html

Agregamos una barra de navegación Bootstrap para moverse entre el listado y el formulario:

```
<nav class="navbar navbar-expand-lg navbar-dark bg-dark mb-4">
  <div class="container">
    <a class="navbar-brand text-warning fw-bold" href="/">🎓 Flask Academy</a>
    <div>
      <a href="/" class="btn btn-outline-warning btn-sm me-2">Listado</a>
      <a href="/agregar" class="btn btn-warning btn-sm">Agregar Curso</a>
    </div>
  </div>
</nav>
```

Coloca este bloque justo antes del `<div class="container py-5">` en index.html.

Prueba rápida

1. Ejecuta:
2. `python app.py`
3. En el navegador:
 - Abre `http://127.0.0.1:5000/` → haz clic en “Agregar Curso”.
 - Llena el formulario con datos válidos.
 - Pulsa **Guardar**.

4. Serás redirigido al listado, y el nuevo curso debe aparecer automáticamente 🎓.

5 Solución a errores comunes

Error	Causa probable	Solución
<code>ModuleNotFoundError: No module named 'wtforms'</code>	Falta dependencia	Ejecuta <code>pip install flask-wtf</code>
<code>RuntimeError: A secret key is required to use CSRF</code>	No configuraste <code>SECRET_KEY</code>	Verifica tu archivo <code>.env</code>
<code>ValueError: could not convert string to float</code>	Se ingresó texto en el campo de duración	Usa un número válido (por ejemplo 12.5)

6 Explicación final

- **Flask-WTF** maneja formularios de forma segura, con validación y token CSRF.
- La lógica de inserción queda en `curso_service.py`, manteniendo limpio `app.py`.
- Los templates usan **Bootstrap modo oscuro** con navegación coherente.
- La app ya implementa el **primer caso completo del CRUD** ✅.

Cuando confirmes que puedes **agregar cursos exitosamente** y verlos **listados**, responde con “listo” para avanzar a la **FASE 6: CASO EDITAR (UPDATE)** ✎

Dijiste:

listo vamos a la fase 6

ChatGPT dijo:

Excelente 🤖

Avanzamos a la **FASE 6: CASO EDITAR (UPDATE)** ✎

En esta fase aprenderás cómo modificar cursos existentes reutilizando el mismo formulario de agregar, pero precargado con los datos del curso.

🧩 FASE 6: CASO EDITAR (UPDATE)

1 Objetivo

- Reutilizar el formulario de Flask-WTF (CursoForm) para editar un curso existente.
- Crear una nueva plantilla `editar_curso.html`.
- Implementar la función `editar_curso()` en la capa de servicios.
- Agregar el botón **Editar** en la tabla del listado.
- Validar que los cambios se reflejen en la base de datos.

2 Archivos a crear o modificar

A) (Nuevo)

 C:\Cursos\Flask\FlaskAcademy\flask_academy\templates\editar_curso.html

```
<!doctype html>
<html lang="es" data-bs-theme="dark">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Editar Curso - Flask Academy</title>

    <link
      href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet"
      integrity="sha384-
QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMHjY6hW+ALEwIH"
      crossorigin="anonymous"
    >
  </head>
  <body>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark mb-4">
      <div class="container">
        <a class="navbar-brand text-warning fw-bold" href="/">🏠 Flask Academy</a>
        <div>
          <a href="/" class="btn btn-outline-warning btn-sm me-2">Listado</a>
          <a href="/agregar" class="btn btn-warning btn-sm">Agregar Curso</a>
        </div>
      </div>
    </nav>

    <div class="container">
      <div class="card border-0 shadow">
        <div class="card-body">
          <h2 class="text-warning mb-4">✏️ Editar Curso</h2>

          <form method="POST">
            {{ form.hidden_tag() }}

            <div class="mb-3">
```

```

        {{ form.nombre.label(class="form-label") }}
        {{ form.nombre(class="form-control", placeholder="Nombre del curso") }}
    </div>

    <div class="mb-3">
        {{ form.instructor.label(class="form-label") }}
        {{ form.instructor(class="form-control", placeholder="Instructor") }}
    </div>

    <div class="mb-3">
        {{ form.duracion.label(class="form-label") }}
        {{ form.duracion(class="form-control", placeholder="Duración en horas")
    }}

    </div>

    <button type="submit" class="btn btn-warning">Actualizar</button>
    <a href="/" class="btn btn-secondary ms-2">Cancelar</a>
</form>
</div>
</div>
</div>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous">
    </script>
</body>
</html>

```

B) (Modificar)

 C:\Cursos\Flask\FlaskAcademy\flask_academy\services\curso_service.py

Agregamos las funciones para **buscar y editar** cursos:

```

from extensions import db
from models import Curso

def obtener_todos():
    return Curso.query.all()

def agregar_curso(nombre, instructor, duracion):
    nuevo_curso = Curso(nombre=nombre, instructor=instructor, duracion=duracion)
    db.session.add(nuevo_curso)
    db.session.commit()

# =====
# 🔍 Buscar curso por ID
# =====
def buscar_por_id(id_curso):
    return Curso.query.get(id_curso)

```

```
# =====
# ✎ Editar curso existente
# =====
def editar_curso(id_curso, nombre, instructor, duracion):
    curso = Curso.query.get(id_curso)
    if curso:
        curso.nombre = nombre
        curso.instructor = instructor
        curso.duracion = duracion
        db.session.commit()
```

C) (Modificar)

 C:\Cursos\Flask\FlaskAcademy\flask_academy\app.py

Agregamos la nueva ruta `/editar/<int:id_curso>` para mostrar y procesar el formulario de edición:

```
import os
from flask import Flask, render_template, redirect, url_for, flash
from dotenv import load_dotenv
from extensions import db, migrate
from models import Curso
from services import curso_service
from forms import CursoForm

load_dotenv()

def create_app():
    app = Flask(__name__)
    app.config["SECRET_KEY"] = os.getenv("SECRET_KEY", "dev-secret-change-me")

    usuario = os.getenv("DB_USER")
    password = os.getenv("DB_PASSWORD")
    servidor = os.getenv("DB_HOST")
    puerto = os.getenv("DB_PORT")
    base_datos = os.getenv("DB_NAME")

    app.config["SQLALCHEMY_DATABASE_URI"] = (
        f"mysql+pymysql://{usuario}:{password}@{servidor}:{puerto}/{base_datos}"
    )
    app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

    db.init_app(app)
    migrate.init_app(app, db)

    @app.route("/")
    def index():
        cursos = curso_service.obtener_todos()
        return render_template("index.html", cursos=cursos)

    @app.route("/agregar", methods=["GET", "POST"])
    def agregar():
```



```

form = CursoForm()
if form.validate_on_submit():
    curso_service.agregar_curso(
        form.nombre.data,
        form.instructor.data,
        form.duracion.data
    )
    flash("✅ Curso agregado exitosamente", "success")
    return redirect(url_for("index"))
return render_template("agregar_curso.html", form=form)

# Nueva ruta para editar curso
@app.route("/editar/<int:id_curso>", methods=["GET", "POST"])
def editar(id_curso):
    curso = curso_service.buscar_por_id(id_curso)
    if not curso:
        flash("⚠️ Curso no encontrado", "warning")
        return redirect(url_for("index"))

    form = CursoForm(obj=curso) # precarga los datos en el formulario

    if form.validate_on_submit():
        curso_service.editar_curso(
            id_curso,
            form.nombre.data,
            form.instructor.data,
            form.duracion.data
        )
        flash("✏️ Curso actualizado correctamente", "success")
        return redirect(url_for("index"))

    return render_template("editar_curso.html", form=form)

return app

if __name__ == "__main__":
    app = create_app()
    app.run(debug=True)

```

💡 Explicación:

- `CursoForm(obj=curso)` precarga los datos en el formulario.
- Si el usuario modifica y guarda, se actualiza el registro en la base de datos.
- Se usa `flash()` para mostrar mensajes de confirmación o error.

D) (Modificar)

📄 C:\Cursos\Flask\FlaskAcademy\flask_academy\templates\index.html

Agregamos un **botón Editar** junto a cada curso.

Busca el bloque dentro de la tabla y reemplázalo así:

```
<tbody>
  {% for curso in cursos %}
    <tr>
      <td>{{ curso.id_curso }}</td>
      <td>{{ curso.nombre }}</td>
      <td>{{ curso.instructor }}</td>
      <td>{{ curso.duracion }}</td>
      <td>
        <a href="{% url_for('editar', id_curso=curso.id_curso) %}" class="btn btn-sm btn-
outline-warning">Editar</a>
      </td>
    </tr>
  {% endfor %}
</tbody>
```

Y actualiza los encabezados de la tabla:

```
<thead>
  <tr class="text-warning">
    <th>ID</th>
    <th>Curso</th>
    <th>Instructor</th>
    <th>Duración (hrs)</th>
    <th>Acciones</th>
  </tr>
</thead>
```

3 Prueba rápida

1. Ejecuta:
2. `python app.py`
3. Abre el navegador en
👉 `http://127.0.0.1:5000/`
4. Haz clic en **Editar** junto a un curso.
 - Se abrirá el formulario con los datos precargados.
 - Modifica cualquier campo y guarda.
 - Regresarás al listado con el curso actualizado. 🎉

4 Solución a errores comunes

Error	Causa probable	Solución
<code>TypeError: editar_curso() missing 1 required positional argument</code>	Parámetro mal definido	Revisa la función <code>editar_curso(id_curso, ...)</code> en <code>curso_service.py</code>
No se actualizan los datos	Falta <code>db.session.commit()</code>	Verifica que esté en la función <code>editar_curso()</code>
No se precargan los valores	Falta <code>obj=curso</code> en el formulario	Corrige en <code>form = CursoForm(obj=curso)</code>

5 Explicación final

- La **misma clase de formulario** sirve tanto para crear como para editar.
- Se mantiene la **separación de responsabilidades**:
 - `app.py` → gestiona rutas y vistas.
 - `curso_service.py` → contiene la lógica de negocio.
 - `forms.py` → maneja validaciones.
 - `templates/` → interfaz del usuario.
- El flujo de actualización queda completamente funcional.

✅ Con esto finalizamos la **FASE 6: CASO EDITAR (UPDATE)**.

Ya puedes modificar cursos existentes con una interfaz moderna y validaciones seguras.

Cuando confirmes que puedes **editar correctamente un curso y ver el cambio reflejado**, responde “listo” para pasar a la **FASE 7: CASO ELIMINAR (DELETE)** 🗑️

Dijiste:

listo vamos a la fase 7

ChatGPT dijo:

Excelente 🎯

Hemos llegado a la **FASE 7: CASO ELIMINAR (DELETE)** 🗑️

Aquí completaremos el CRUD implementando la opción para eliminar cursos, con una confirmación elegante mediante **Bootstrap modal**.

🧩 FASE 7: CASO ELIMINAR (DELETE)

1 Objetivo

- Agregar la función `eliminar_curso()` en la capa de servicios.
- Implementar un botón **Eliminar** en la tabla principal.
- Mostrar una **alerta modal** para confirmar la eliminación.
- Refrescar automáticamente la tabla tras borrar el registro.

2 Archivos a crear o modificar

A) (Modificar)

 C:\Cursos\Flask\FlaskAcademy\flask_academy\services\curso_service.py

Agregamos la nueva función de eliminación:

```
from extensions import db
from models import Curso

def obtener_todos():
    return Curso.query.all()

def agregar_curso(nombre, instructor, duracion):
    nuevo_curso = Curso(nombre=nombre, instructor=instructor, duracion=duracion)
    db.session.add(nuevo_curso)
    db.session.commit()

def buscar_por_id(id_curso):
    return Curso.query.get(id_curso)

def editar_curso(id_curso, nombre, instructor, duracion):
    curso = Curso.query.get(id_curso)
    if curso:
        curso.nombre = nombre
        curso.instructor = instructor
        curso.duracion = duracion
        db.session.commit()

# =====
#  Eliminar curso
# =====
def eliminar_curso(id_curso):
    curso = Curso.query.get(id_curso)
    if curso:
        db.session.delete(curso)
        db.session.commit()
```

💡 Esta función busca el curso por su ID y lo elimina definitivamente de la base de datos.

B) (Modificar)

📄 C:\Cursos\Flask\FlaskAcademy\flask_academy\app.py

Agregamos la ruta /eliminar/<int:id_curso> con confirmación:

```
import os
from flask import Flask, render_template, redirect, url_for, flash
from dotenv import load_dotenv
from extensions import db, migrate
from models import Curso
from services import curso_service
from forms import CursoForm

load_dotenv()

def create_app():
    app = Flask(__name__)
    app.config["SECRET_KEY"] = os.getenv("SECRET_KEY", "dev-secret-change-me")

    usuario = os.getenv("DB_USER")
    password = os.getenv("DB_PASSWORD")
    servidor = os.getenv("DB_HOST")
    puerto = os.getenv("DB_PORT")
    base_datos = os.getenv("DB_NAME")

    app.config["SQLALCHEMY_DATABASE_URI"] = (
        f"mysql+pymysql://{usuario}:{password}@{servidor}:{puerto}/{base_datos}"
    )
    app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

    db.init_app(app)
    migrate.init_app(app, db)

    @app.route("/")
    def index():
        cursos = curso_service.obtener_todos()
        return render_template("index.html", cursos=cursos)

    @app.route("/agregar", methods=["GET", "POST"])
    def agregar():
        form = CursoForm()
        if form.validate_on_submit():
            curso_service.agregar_curso(
                form.nombre.data,
                form.instructor.data,
                form.duracion.data
            )
            flash("✅ Curso agregado exitosamente", "success")
            return redirect(url_for("index"))
```

```

    return render_template("agregar_curso.html", form=form)

@app.route("/editar/<int:id_curso>", methods=["GET", "POST"])
def editar(id_curso):
    curso = curso_service.buscar_por_id(id_curso)
    if not curso:
        flash("⚠ Curso no encontrado", "warning")
        return redirect(url_for("index"))

    form = CursoForm(obj=curso)
    if form.validate_on_submit():
        curso_service.editar_curso(
            id_curso,
            form.nombre.data,
            form.instructor.data,
            form.duracion.data
        )
        flash("✏ Curso actualizado correctamente", "success")
        return redirect(url_for("index"))
    return render_template("editar_curso.html", form=form)

# Nueva ruta para eliminar curso
@app.route("/eliminar/<int:id_curso>")
def eliminar(id_curso):
    curso = curso_service.buscar_por_id(id_curso)
    if not curso:
        flash("⚠ Curso no encontrado", "warning")
    else:
        curso_service.eliminar_curso(id_curso)
        flash(f"🗑 Curso '{curso.nombre}' eliminado exitosamente", "success")
    return redirect(url_for("index"))

return app

if __name__ == "__main__":
    app = create_app()
    app.run(debug=True)

```

C) (Modificar)

📄 C:\Cursos\Flask\FlaskAcademy\flask_academy\templates\index.html

Agregamos el botón **Eliminar** y la ventana modal de confirmación.

Busca el bloque <tbody> y reemplázalo por este:

```

<tbody>
  {% for curso in cursos %}
  <tr>
    <td>{{ curso.id_curso }}</td>
    <td>{{ curso.nombre }}</td>
    <td>{{ curso.instructor }}</td>
    <td>{{ curso.duracion }}</td>

```

```

<td>
  <a href="{ url_for('editar', id_curso=curso.id_curso) }}" class="btn btn-sm btn-
outline-warning me-2">Editar</a>

  <!-- Botón que activa el modal -->
  <button type="button" class="btn btn-sm btn-outline-danger" data-bs-
toggle="modal" data-bs-target="#modalEliminar{{ curso.id_curso }}">
    Eliminar
  </button>

  <!-- Modal de confirmación -->
  <div class="modal fade" id="modalEliminar{{ curso.id_curso }}" tabindex="-1"
aria-hidden="true">
    <div class="modal-dialog modal-dialog-centered">
      <div class="modal-content bg-dark text-white">
        <div class="modal-header border-0">
          <h5 class="modal-title text-warning">Confirmar eliminación</h5>
          <button type="button" class="btn-close btn-close-white" data-bs-
dismiss="modal"></button>
        </div>
        <div class="modal-body">
          ¿Seguro que deseas eliminar el curso <strong>{{ curso.nombre }}</strong>?
        </div>
        <div class="modal-footer border-0">
          <button type="button" class="btn btn-secondary" data-bs-
dismiss="modal">Cancelar</button>
          <a href="{ url_for('eliminar', id_curso=curso.id_curso) }}" class="btn
btn-danger">Eliminar</a>
        </div>
      </div>
    </div>
  </div>
</td>
</tr>
{% endfor %}
</tbody>

```

Y asegúrate de que la tabla tenga la columna **Acciones** en el <thead>:

```

<thead>
  <tr class="text-warning">
    <th>ID</th>
    <th>Curso</th>
    <th>Instructor</th>
    <th>Duración (hrs)</th>
    <th>Acciones</th>
  </tr>
</thead>

```

3 Prueba rápida

1. Ejecuta:
2. `python app.py`

3. Abre el navegador en
 `http://127.0.0.1:5000/`
4. Haz clic en **Eliminar** junto a un curso.
 - Se abrirá un modal de confirmación.
 - Si confirmas, el curso será eliminado y verás un mensaje de éxito.
 - La tabla se actualizará automáticamente al volver al listado.

4 Solución a errores comunes

Error	Causa probable	Solución
<code>IntegrityError</code>	Intentas eliminar un curso relacionado con otra tabla (en proyectos futuros)	Implementar control de dependencias o relaciones
No aparece el modal	Faltó el script de Bootstrap	Asegúrate de tener el bloque <code><script></code> al final del HTML
No se elimina el curso	Enlace incorrecto o error en la ruta	Verifica que <code>url_for('eliminar', id_curso=curso.id_curso)</code> esté correcto

5 Explicación final

- El botón **Eliminar** no borra directamente: primero abre un **modal de confirmación** para evitar errores.
- Si el usuario confirma, se redirige a la ruta `/eliminar/<id>` que llama a `curso_service.eliminar_curso()`.
- Todo se mantiene modular:
 - `curso_service.py` → lógica de negocio.
 - `app.py` → control de flujo.
 - `index.html` → interfaz con Bootstrap.
- Con esto, el **CRUD queda 100% completo y funcional** 🎓

✓ Conclusión general del proyecto

Has creado un **sistema CRUD completo con Flask, Bootstrap y MySQL**, aplicando una arquitectura profesional con capas separadas:

- `app.py` → rutas y configuración
- `models.py` → modelo de datos
- `services/` → lógica de negocio

- templates/ → vistas
- forms.py → validaciones seguras con Flask-WTF

Este enfoque modular es escalable, mantenible y ideal para proyectos reales.

Uso de una plantilla base con Flask y Jinja:

Vamos a refactorizar tu aplicación para usar un **layout base con herencia de plantillas**, de forma profesional y moderna.

Así todos los archivos (index.html, agregar_curso.html, editar_curso.html) compartirán el mismo diseño, cabecera y pie.

A continuación tienes **todos los archivos completos**, listos para copiar y pegar.

El resultado será idéntico al de tus imágenes, pero con una estructura más limpia y escalable 



1. templates/base.html

```
<!doctype html>
<html lang="es" data-bs-theme="dark">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>{% block title %}Flask Academy{% endblock %}</title>

    <!-- Bootstrap y Bootstrap Icons -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
    <link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-
icons.css" rel="stylesheet">
  </head>

  <body class="bg-dark text-light">

    <!-- Navbar global -->
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark border-bottom border-
warning mb-4">
      <div class="container">
        <a class="navbar-brand text-warning fw-bold" href="/">
          <i class="bi bi-mortarboard-fill me-2"></i> Flask Academy
        </a>
```

```

    <div class="ms-auto">
        {% block navbar_buttons %}
        <!-- Se define individualmente en cada página -->
        {% endblock %}
    </div>
</div>
</nav>

<!-- Contenido principal -->
<main class="container">
    {% block content %}
    {% endblock %}
</main>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></sc
ript>
</body>
</html>

```



2. templates/index.html

```

{% extends "base.html" %}

{% block title %}Flask Academy - Listado de Cursos{% endblock %}

{% block navbar_buttons %}
<a href="/agregar" class="btn btn-warning fw-semibold">
    <i class="bi bi-plus-lg me-1"></i> Agregar Curso
</a>
{% endblock %}

{% block content %}
<h2 class="text-center text-warning mb-4 fw-bold">Listado de cursos</h2>

<div class="table-responsive">
    <table class="table table-dark table-hover align-middle text-center">
        <thead class="table-dark border-bottom border-warning">
            <tr class="text-warning">
                <th>ID</th>
                <th>Nombre</th>
                <th>Instructor</th>
                <th>Duración (hrs)</th>
                <th>Acciones</th>
            </tr>
        </thead>
        <tbody>
            {% for curso in cursos %}
            <tr>
                <td>{{ curso.id_curso }}</td>
                <td>{{ curso.nombre }}</td>
                <td>{{ curso.instructor }}</td>
                <td>{{ "%.2f"|format(curso.duracion) }}</td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</div>

```

```

<td>
    <a href="{% url_for('editar', id_curso=curso.id_curso) %}" class="btn btn-sm
btn-warning me-1">
        <i class="bi bi-pencil-fill"></i> Editar
    </a>

    <button class="btn btn-sm btn-outline-danger" data-bs-toggle="modal" data-bs-
target="#modalEliminar{% curso.id_curso %}">
        <i class="bi bi-trash-fill"></i> Eliminar
    </button>

    <!-- Modal de confirmación -->
    <div class="modal fade" id="modalEliminar{% curso.id_curso %}" tabindex="-1"
aria-hidden="true">
        <div class="modal-dialog modal-dialog-centered">
            <div class="modal-content bg-dark text-light border border-warning">
                <div class="modal-header border-0">
                    <h5 class="modal-title text-warning">
                        <i class="bi bi-exclamation-triangle-fill me-2"></i> Confirmar
eliminación
                    </h5>
                    <button type="button" class="btn-close btn-close-white" data-bs-
dismiss="modal"></button>
                </div>
                <div class="modal-body">
                    ¿Seguro que deseas eliminar el curso <strong>{% curso.nombre
%}</strong>?
                </div>
                <div class="modal-footer border-0">
                    <button class="btn btn-secondary" data-bs-
dismiss="modal">Cancelar</button>
                    <a href="{% url_for('eliminar', id_curso=curso.id_curso) %}"
class="btn btn-danger">
                        <i class="bi bi-trash3-fill me-1"></i> Eliminar
                    </a>
                </div>
            </div>
        </div>
    </div>
</td>
</tr>
{% else %}
<tr>
    <td colspan="5" class="text-muted">No hay cursos registrados.</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
{% endblock %}

```



3. templates/agregar_curso.html

```
{% extends "base.html" %}
```

```
{% block title %}Agregar Curso - Flask Academy{% endblock %}

{% block navbar_buttons %}
<a href="/" class="btn btn-outline-warning fw-semibold">
  <i class="bi bi-arrow-left-circle me-1"></i> Volver
</a>
{% endblock %}

{% block content %}
<h2 class="text-center text-warning mb-4 fw-bold">Registrar nuevo curso</h2>

<form method="POST" class="mx-auto" style="max-width: 600px;">
  {{ form.hidden_tag() }}

  <div class="mb-3">
    {{ form.nombre.label(class="form-label text-warning") }}
    {{ form.nombre(class="form-control bg-dark text-light border-warning",
placeholder="Ejemplo: Python desde Cero") }}
  </div>

  <div class="mb-3">
    {{ form.instructor.label(class="form-label text-warning") }}
    {{ form.instructor(class="form-control bg-dark text-light border-warning",
placeholder="Ejemplo: Marcela Gamiño") }}
  </div>

  <div class="mb-4">
    {{ form.duracion.label(class="form-label text-warning") }}
    {{ form.duracion(class="form-control bg-dark text-light border-warning",
placeholder="Ejemplo: 15.0") }}
  </div>

  <div class="text-center">
    <button type="submit" class="btn btn-warning fw-semibold">
      <i class="bi bi-save2-fill me-1"></i> Guardar curso
    </button>
  </div>
</form>
{% endblock %}
```



4. templates/editar_curso.html

```
{% extends "base.html" %}

{% block title %}Editar Curso - Flask Academy{% endblock %}

{% block navbar_buttons %}
<a href="/" class="btn btn-outline-warning fw-semibold">
  <i class="bi bi-arrow-left-circle me-1"></i> Volver
</a>
{% endblock %}

{% block content %}
```

```

<h2 class="text-center text-warning mb-4 fw-bold">Editar curso</h2>

<form method="POST" class="mx-auto" style="max-width: 600px;">
    {{ form.hidden_tag() }}

    <div class="mb-3">
        {{ form.nombre.label(class="form-label text-warning") }}
        {{ form.nombre(class="form-control bg-dark text-light border-warning") }}
    </div>

    <div class="mb-3">
        {{ form.instructor.label(class="form-label text-warning") }}
        {{ form.instructor(class="form-control bg-dark text-light border-warning") }}
    </div>

    <div class="mb-4">
        {{ form.duracion.label(class="form-label text-warning") }}
        {{ form.duracion(class="form-control bg-dark text-light border-warning") }}
    </div>

    <div class="text-center">
        <button type="submit" class="btn btn-warning fw-semibold">
            <i class="bi bi-save2-fill me-1"></i> Guardar curso
        </button>
    </div>
</form>
{% endblock %}

```

✅ Ventajas de esta versión:

- Diseño uniforme en todas las páginas (modo oscuro, íconos, tipografía, botones).
- La barra de navegación se define solo una vez en `base.html`.
- Fácil mantenimiento y ampliación (por ejemplo, si agregas más secciones).
- Estilo idéntico al mostrado en tus capturas, con bordes amarillos, fondo oscuro y tipografía moderna.

Consultas extra con ChatGPT del curso:

Explicame a más detalle el archivo `app.py`:

¡Claro! Te lo explico paso a paso y con contexto para que entiendas **qué hace cada línea**, **por qué está así**, y **cómo extenderlo** más adelante. 😊

Qué hace cada bloque

1. Imports

- `import os`: te permite leer variables de entorno (como `SECRET_KEY`) con `os.getenv`.
- `from flask import Flask, render_template`: crea la app Flask y renderiza plantillas HTML.
- `from dotenv import load_dotenv`: carga las variables del archivo `.env` a las variables de entorno del proceso.

2. Carga del `.env`

- `load_dotenv()` lee un archivo `.env` (si existe en la raíz del proyecto) y “exporta” esas claves al entorno.
Útil para no hardcodear secretos o credenciales en el código.

3. Patrón “Application Factory”

- `def create_app()`: define una **fábrica de aplicación**. En lugar de tener una app global, creas la app bajo demanda.
Ventajas:
 - Facilita pruebas (puedes crear múltiples instancias con distinta config).
 - Escala mejor con *blueprints* y configuraciones por entorno.

4. Instancia de Flask

- `app = Flask(__name__)`: crea la app. `__name__` ayuda a Flask a ubicar rutas relativas (por ejemplo, la carpeta `templates/`).

5. Configuración de seguridad

- `app.config["SECRET_KEY"] = os.getenv("SECRET_KEY", "dev-secret-change-me")`
 - `SECRET_KEY` cifra cookies de sesión y habilita protección **CSRF** (cuando uses Flask-WTF).
 - Se intenta leer de la variable de entorno `SECRET_KEY`; si no existe, usa el valor por defecto `"dev-secret-change-me"`.
 - En **desarrollo** es práctico tener un default; en **producción** debes **poner una clave larga, aleatoria y secreta** en el `.env` (y nunca subir ese `.env` a Git).

6. Ruta principal

- `@app.route("/")` define la URL `/`.
- `def index(): return render_template("index.html")`
Renderiza `templates/index.html`. Aún no hay lógica de negocio, solo la vista base.

7. Ejecución directa (PyCharm/CLI)

- El bloque:
- `if __name__ == "__main__":`
- `app = create_app()`

- `app.run(debug=True)`

permite lanzar la app con el botón **Run** de PyCharm o con `python app.py`.

- `debug=True` recarga en caliente y muestra el *debugger*. No lo uses en producción.

Flujo de ejecución (resumen mental)

- El intérprete carga el módulo → `load_dotenv()` prepara variables.
- Al **ejecutar directamente** el archivo, entra al `if __name__ == "__main__":`
 - Llama `create_app()` → instancia Flask, configura `SECRET_KEY`, registra la ruta `/`.
 - `app.run(debug=True)` levanta el servidor de desarrollo.
- Cuando un usuario visita `/`, Flask busca y renderiza `templates/index.html`.

Estructura mínima de carpetas esperada

```
tu_proyecto/
├── app.py           # (el archivo que mostraste)
├── .env             # variables de entorno (no subir a Git)
└── templates/
    └── index.html
```

Si `index.html` no está en `templates/`, Flask dará error `TemplateNotFound`.

Ejemplos rápidos que te sirven ya

1) Contenido típico de `.env` (desarrollo):

```
SECRET_KEY=una_clave_larga_aleatoria_que_no_subas_a_github
```

2) Generar una `SECRET_KEY` aleatoria desde Python (una vez):

```
import secrets
print(secrets.token_hex(32))
```

Copia ese valor a tu `.env`.

3) Un `index.html` mínimo (en `templates/index.html`):

```
<!doctype html>
<html lang="es">
  <head>
    <meta charset="utf-8">
    <title>FlaskAcademy</title>
  </head>
  <body>
    <h1>¡Hola, FlaskAcademy!</h1>
    <p>Plantilla base lista.</p>
  </body>
</html>
```

Por qué es importante `SECRET_KEY` (y cómo se conecta con Flask-WTF)

- **Sesiones:** Flask firma (y opcionalmente cifra) la cookie de sesión; sin `SECRET_KEY`, cualquiera podría manipularla.
 - **CSRF:** cuando uses formularios con Flask-WTF, cada `<form>` incluirá un token CSRF. Flask-WTF valida ese token usando la `SECRET_KEY`. Sin una clave estable y secreta, los formularios fallarán o serás vulnerable a ataques CSRF.
-

Errores comunes (y cómo evitarlos)

- **TemplateNotFound: index.html**
Solución: crea `templates/index.html` y verifica que la carpeta se llame exactamente `templates`.
 - **No se toma la `SECRET_KEY` del `.env`**
 - Asegúrate de que `.env` esté en la **raíz** del proyecto (junto a `app.py`) y que se ejecute `load_dotenv()` **antes** de leer `os.getenv`.
 - Reinicia el intérprete/servidor tras crear o cambiar `.env`.
 - **Subir `.env` a Git (peligro)**
 - Agrega `.env` a tu `.gitignore`.
 - **Usar `debug=True` en producción**
 - Evítalo. En producción usa un servidor WSGI (ej. Gunicorn/Waitress) y `debug=False`.
-

Cómo escalar este archivo de forma limpia

- **Config por clase** (opcional cuando crezca tu app):
- `class Config:`
- `SECRET_KEY = os.getenv("SECRET_KEY", "dev-secret-change-me")`

- - `def create_app():`
 - `app = Flask(__name__)`
 - `app.config.from_object(Config)`
 - `# registrar blueprints aquí`
 - `return app`
 - **Blueprints para separar módulos (por ejemplo, cursos, usuarios):**
 - Crea `cursos/__init__.py` y `cursos/routes.py` con un Blueprint.
 - Regístralo en `create_app()`:
 - `from cursos.routes import cursos_bp`
 - `app.register_blueprint(cursos_bp, url_prefix="/cursos")`
 - **Cargar más config del `.env` (cuando agregues DB):**
 - `app.config["MYSQL_USER"] = os.getenv("MYSQL_USER")`
 - `app.config["MYSQL_PASSWORD"] = os.getenv("MYSQL_PASSWORD")`
 - `app.config["MYSQL_DB"] = os.getenv("MYSQL_DB")`
 - `app.config["MYSQL_HOST"] = os.getenv("MYSQL_HOST", "localhost")`
-

Cómo ejecutarlo (rápido)

- **PyCharm:** botón *Run* (gracias al bloque `if __name__ == "__main__":`).
- **CLI:**
- `python app.py`

o con el runner de Flask:

```
set FLASK_APP=app.py           # Windows CMD
set FLASK_ENV=development
flask run
```

Checklist mental para tu proyecto

- `.env` creado con `SECRET_KEY` (no subirlo a Git).
- Carpeta `templates/` con `index.html`.
- `load_dotenv()` antes de leer `os.getenv`.
- `debug=True` solo en desarrollo.
- Preparado para escalar con `factory` + `blueprints`.

Si quieres, te muestro **cómo integrar Flask-WTF** en este esqueleto (formulario con CSRF funcionando) y dejamos todo listo para tu Fase de formularios. ¿Le damos? 🙌

Dijiste:

que es ORM cuando trabajamos con `flask-sqlalchemy`?

ChatGPT dijo:

Excelente pregunta 😊

Cuando trabajas con **Flask-SQLAlchemy**, estás usando un **ORM** (Object-Relational Mapper), o en español, **Mapeador Objeto-Relacional**.

Te explico con detalle qué significa y por qué es tan útil 🙌



¿Qué es un ORM?

Un **ORM** es una capa intermedia entre tu código Python (orientado a objetos) y la base de datos relacional (orientada a tablas).

Su función es **traducir automáticamente** entre ambos mundos:

En Python (objeto) En Base de Datos (tabla)

Clase <code>Curso</code>	Tabla <code>curso</code>
Atributo <code>nombre</code>	Columna <code>nombre</code>
Instancia <code>curso1</code>	Fila (registro)

Esto te permite trabajar con **clases y objetos**, sin escribir directamente consultas SQL cada vez que quieras insertar, modificar o eliminar datos.



¿Cómo funciona en Flask-SQLAlchemy?

Flask-SQLAlchemy es una extensión que integra **SQLAlchemy** (el ORM más popular de Python) con **Flask**. Cuando defines un modelo en tu aplicación Flask, **SQLAlchemy** crea el **mapeo** entre la clase y la tabla en la base de datos.

Por ejemplo:

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

class Curso(db.Model):
    __tablename__ = 'curso'
    id = db.Column(db.Integer, primary_key=True)
    nombre = db.Column(db.String(100))
    instructor = db.Column(db.String(100))
    duracion_horas = db.Column(db.Integer)
```

Aquí ocurre la **magia del ORM**:

- Curso ↔ tabla curso
 - id, nombre, instructor, duracion_horas ↔ columnas de la tabla
 - Cada objeto Curso representa una **fila** de la tabla
-

💡 Ejemplo práctico

Sin ORM (puro SQL):

```
cursor.execute("INSERT INTO curso (nombre, instructor, duracion_horas) VALUES (?, ?, ?)",  
              ("Python Básico", "Marce", 20))
```

Con ORM:

```
nuevo_curso = Curso(nombre="Python Básico", instructor="Marce", duracion_horas=20)  
db.session.add(nuevo_curso)  
db.session.commit()
```

👉 El ORM se encarga de traducir ese `add()` y `commit()` a la instrucción SQL real que entiende la base de datos.

📋 Ventajas del ORM

- ✅ **Más legible:** trabajas con objetos y métodos, no con strings SQL.
 - ✅ **Menos errores:** SQLAlchemy valida tipos y nombres automáticamente.
 - ✅ **Portabilidad:** puedes cambiar de motor (MySQL, PostgreSQL, SQLite) sin reescribir el código.
 - ✅ **Seguridad:** evita inyección SQL, ya que los parámetros se manejan internamente.
 - ✅ **Integración natural:** puedes acceder a relaciones (`curso.profesor.nombre`) sin hacer JOINS manuales.
-

⚠️ Cuándo usarlo y cuándo no

Úsalo si:

- Tu proyecto tiene lógica de negocio moderada o compleja.
- Prefieres trabajar con objetos y relaciones entre ellos.

Evítalo (o combínalo con SQL manual) si:

- Necesitas consultas extremadamente optimizadas y específicas.
 - Trabajas con datos masivos donde el ORM genera sobrecarga.
-

En resumen

El ORM de Flask-SQLAlchemy:

- Te permite **definir tus tablas como clases Python**.
- Convierte automáticamente tus objetos a registros SQL y viceversa.
- Hace tu código más **limpio, seguro y mantenible**.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)